Nearby errands: mathematical specification

Revised 14 October 2026

Source transcription, revised 2026-10-14. This specifies the current feature's matching decisions and state transitions. The mathematical model includes recognized tags, task eligibility, pauses, selected origins, point distances and coverage; existing ordering and presentation definitions are retained for source correspondence. Catalogue records are input data; shared Yjs, rich-text, JSON, hashing, browser and generic CRUD semantics are named primitives. It is not a formal equivalence proof. The companion `.tex` file typesets this document.

1. Notation and inputs

$\perp$ means absent; ϵ is the empty sequence. $q \parallel r$ concatenates sequences. $[x \in q : P(x)]$ filters without reordering. $\mathrm{uniq}(q)$ keeps first occurrences; uniq_{id} does so by item ID. $\mathrm{ids}(q)$ is a set. $\mathrm{head}(\epsilon) = \perp$. Sorting is stable, so comparison equality preserves input order. $\prec_c$ is the runtime's `localeCompare` ordering; $\prec_u$ is its UTF-16 string ordering, and string lengths count UTF-16 code units. $\mathbf{1}_P$ is 1 when P holds, otherwise 0. Write $x.lat, x.lon$ for the record fields `latitude,longitude`.

Let I be the stored item sequence, L, F the list/folder maps, B the bundled retail sequence, U the suburb sequence, and X the insertion-ordered custom-location map. Let

$$D = B \parallel \mathrm{suburbLocations}(U) \parallel \mathrm{values}(X),$$
$$\Sigma = (I, L, F, X, Q, P_T, P_I, s, f, p, h; \rho, \eta, v, \lambda, e, \nu).$$

Q maps item IDs to checklist records (ϕ, δ): saved fingerprint and dismissed flag. P_T, P_I are paused tags and item IDs. s is the stored starting-location record; f enables location filtering; p is the pill preview limit; h is the hot-suburb sequence. Local state: radius ρ, historical-view flag η, GPS request generation v, locating flag λ, location error e, navigation state ν. All predicates below are recomputed from the current live or historical snapshot.

The input records are those produced by `readAllItems`, `readLists` and `readFolders`: absent item booleans are false, absent item order is 0, absent parent is $\perp$, price/quantity/timestamps default to $\perp$, and legacy checkbox maps are overlaid by per-checkbox keys (only literal `true` counts). Record identity and remaining fields are retained. Folder parent links are resolved by the same cycle rule as item links below.

2. Tags and destinations

$H(a)$ scans a name from left to right for `#word` tokens preceded by start-of-string or runtime whitespace, lowercases the captured words, and retains duplicates. Here `word` is one or more ASCII letters, digits or underscores, matching the source's `\w`; punctuation terminates it.

For a tag t, trim whitespace, remove one leading `#`, and lowercase to obtain t_0. With $\mathcal{A}$ the map in `src/lib/locations/brand-aliases.json`, overridden by the following three entries, normalize by one lookup:

$$\mathcal{A}[\text{wooloworths}] = \mathcal{A}[\text{woolworth}] = \mathcal{A}[\text{woolies}] = \text{woolworths},$$
$$n(t) = \begin{cases} \mathcal{A}[t_0], & t_0 \in \operatorname{dom} \mathcal{A}, \\ t_0, & \text{otherwise.} \end{cases}$$

Location support, recognized errand tags, and ordered matching tags are

$$
\begin{aligned}
S_0(d) &= \{n(t) : t \in d.tags\},\\
S(d) &= S_0(d) \cup \{\text{supermarket} : S_0(d) \cap \{\text{coles}, \text{woolworths}, \text{aldi}\} \neq \varnothing\}\\
&\quad \cup \{\text{hardware} : S_0(d) \cap \{\text{bunnings}, \text{mitre10}, \text{homehardware}\} \neq \varnothing\}\\
&\quad \cup \{\text{pcparts} : S_0(d) \cap \{\text{scorptec}, \text{centrecom}, \text{cpl}, \text{msy}\} \neq \varnothing\},\\
K_D &= \bigcup_{d \in D} S(d), \qquad G = \{\text{bank}, \text{errand}\}, \qquad K = K_D \cup G,\\
M(t,d) &= [a \in \text{uniq}([n(x) : x \in t]) : a \in S(d)].
\end{aligned}
$$

Conditional singleton sets are empty when their condition is false. Recognition uses all destinations, independent of radius, starting-point status or pauses. Matching needs any supported tag. No shopping-centre, brand-frequency or route preference is added.

K_D is the keyset of the location-to-tag relation, not the set of all hashtags in the user's tasks. A tag outside K cannot independently create an errand candidate. Tags are compared after normalization; multiple tags and multiple locations use existential (OR) matching. Normalization does not perform general spelling correction: `#supermarket` is recognized, whereas `#supermaket` is not unless registered on a location. Ordinary tags such as `#tools` and `#supplies_photos` remain ordinary while absent from K. Registering one on a custom location deliberately makes it a location tag; deleting its last supporting location removes it from K_D.

A shopping-centre hashtag such as `#northland` can support many business records, with their individual or shared centre coordinates. It does not introduce a separate centre polygon or a distance-to-boundary calculation. A custom location can support several hashtags at the same coordinate; three active tasks tagged `#myhouse` each match the saved location supporting that tag.

For each suburb u, its destination has ID `suburb-` concatenated with $u.id$, name $u.name$ followed by `(suburb centre)`, its catalogue coordinates/source, accuracy class `suburb`, and the single tag

$$\text{suburbTag}(u) = \text{keep}_{a\ldots z,0\ldots 9}\big(\text{lower}(\text{strip}_{0300\ldots036f}(\text{NFKD}(u.name)))\big).$$

3. Source order, completion and candidates

Within each list, a missing parent becomes $\perp$. In each parent cycle, the node with smallest ID under $\prec_u$ becomes a root. Sort every sibling sequence by $(order, id_{\prec_u})$ and traverse roots in preorder, visiting a parent before its children. Concatenate lists in their first-occurrence order in I; call the resulting sequence $T(I)$. Resolve/order the full tree before filtering. These repairs affect the read snapshot, not stored parent links.

Discard missing-list items. For an item i whose list exists, let $\ell = L[i.listId]$, $b(i)$ be the named-checkbox sequence of its list's direct folder (empty if absent), and $\text{anc}(\ell)$ be that folder and all existing ancestors, stopping at a missing parent or a repeated ID. Define

$$
\begin{aligned}
\text{arch}(\ell) &= \ell.archived \vee \bigvee_{a \in \text{anc}(\ell)} a.archived,\\
\text{checked}(i) &= \begin{cases} i.checks[\text{last}(b(i)).id] = \text{true}, & |b(i)| > 0,\\ i.checked, & |b(i)| = 0, \end{cases}\\
d_i &= \neg i.note \wedge \text{checked}(i),\\
j_i &\iff \{n(t) : t \in H(i.name)\} \cap K = \varnothing,\\
\tau_i &= \begin{cases} \text{uniq}\big(H(i.name) \parallel H(\ell.name)\big), & j_i,\\ H(i.name), & \neg j_i, \end{cases}\\
k_i &\iff \{n(t) : t \in \tau_i\} \cap K \neq \varnothing,\\
C_b &= [i \in T(I) : \ell \neq \perp \wedge \ell.type \neq \text{divider} \wedge \neg\,\text{arch}(\ell) \wedge \neg i.heading\\
&\qquad\qquad \wedge (b \vee \neg d_i) \wedge \neg(\ell.done \wedge j_i) \wedge k_i],\\
C &= C_{\text{true}}.
\end{aligned}
$$

Each candidate carries (i, ℓ, τ_i, j_i). C_{false} is `collectErrands`' default result; the screen uses C. Notes ignore checked state. Folder names and parent-item tags contribute no inherited tags. Unrecognized item tags are retained alongside list tags, but a candidate requires at least one normalized effective tag in K. Recognized item tags override all list tags; otherwise list tags are inherited. Completing a list excludes its inherited candidates, including notes, while explicitly recognized item errands remain eligible.

For example, `Take photos #supplies_photos` in an untagged list or a list tagged only `#supplies_photos` is excluded when that tag is absent from K. The same task in `Shopping #supermarket` inherits `supermarket` and qualifies. `Buy milk #coles` in that list uses its own location tags instead. Changing D can change K, j_i, k_i and inheritance without editing any task; ordinary hashtags in the original items are never removed by this calculation.

4. Fingerprints, pauses and checklist membership

Let $\mathrm{omit}(i, \texttt{order})$ be a copy of the resolved item record with only the `order` field removed. All other fields and values are retained. Let $\mathrm{text}(i)$ be its current Yjs rich-text delta when a `Y.Text` exists, otherwise its name. Define the hash payload and fingerprint separately:

$$\begin{aligned}\mathrm{payload}(i) = \{&\texttt{item} : \mathrm{omit}(i, \texttt{order}),\ \texttt{listName} : \ell.\texttt{name},\\ &\texttt{done} : d_i,\ \texttt{text} : \mathrm{text}(i)\},\\ \phi_i = \mathrm{hex}_{\mathrm{lower}}&(\mathrm{SHA256}(\mathrm{UTF8}(\mathrm{JSON}(\mathrm{stable}(\mathrm{payload}(i)))))) .\end{aligned}$$

stable recursively reconstructs objects with keys sorted by $\prec_c$ and preserves array order; JSON retains the runtime's property-enumeration/serialization semantics. The names `item`, `listName`, `done` and `text` are literal object keys, not mathematical variables. Equality is actual digest equality. List name is included for every item, including explicit errands; metadata, checkbox state, timestamps, resolved parent and rich-text formatting are included. Item `order` alone is excluded, so changing only that field does not change the fingerprint.

Only Q entries with a 64-character lowercase hexadecimal fingerprint and a Boolean dismissed flag are read as checklist states. Write $q_i = Q[i.id]$ for a valid record; missing/invalid records give $q_i = \bot$. Define

$$\begin{aligned}z_i &\iff q_i \neq \bot \wedge q_i.\delta \wedge (d_i \vee q_i.\phi = \phi_i),\\ a_i &\iff i.id \in P_I \vee \exists t \in \tau_i : n(t) \in \{n(x) : x \in P_T\},\\ o_i &= \mathrm{head}([n(t) : t \in \tau_i,\ n(t) \in G]), \qquad \omega_i \iff o_i \neq \bot,\\ A &= [i \in C : \neg d_i \wedge \neg z_i \wedge \neg a_i],\\ W &= \{i.id : i \in A,\ \exists d \in D : |M(\tau_i, d)| > 0\},\\ E &= [i \in C : \neg z_i \wedge \neg a_i \wedge \big((d_i \wedge q_i \neq \bot) \vee (\neg d_i \wedge (i.id \in W \vee \omega_i \vee q_i \neq \bot))\big)],\\ P &= [i \in C : \neg d_i \wedge \neg z_i \wedge a_i], \qquad E^+ = [i \in E : d_i].\end{aligned}$$

A are active errands; W are active tasks supported anywhere in the database; E is the persistent checklist; P is the paused list; E^+ are clearable completed rows. Computing W does not validate coordinates or impose a radius. Remembered state does not override membership in C: losing every recognized effective tag excludes an item from E, A and P, even if its checklist or pause record remains stored. A remembered candidate may remain in E without a nearby match, and optional tags qualify without a database match. An already-completed task without a remembered state is excluded from E. Optional tags apply to the whole task even if it also has other tags; the first optional tag in tag order supplies its label.

5. Starting point, distance and nearby stops

The saved selectable-origin sets are

$$O_U(h) = \{u \in U : u.id \in h\}, \qquad O_X = \{x \in \mathrm{values}(X) : x.\mathit{startingPoint} = \mathrm{true}\}.$$

GPS supplies a separate measured origin. h is stored independently of destination recognition, but O_X uses flags on the same custom records used as destinations. There is no general independent origin catalogue referencing arbitrary records in B: a bundled shopping centre must be represented by an enabled custom location to become a saved selectable origin. Origin tags do not restrict destination tags; selecting a `myhouse` origin still permits matching `supermarket` or `northland` destinations.

Resolve the stored starting point s as follows. Invalid/absent records yield $\bar{s} = \bot$; GPS/suburb records yield themselves; custom records yield themselves with label/coordinates replaced by the referenced X record exactly when it exists and has `startingPoint =true`, otherwise $\bot$.

$$
\begin{aligned}
\operatorname{valid}(x) &\iff \operatorname{finite}(x.lat) \wedge \operatorname{finite}(x.lon) \wedge |x.lat| \le 90 \wedge |x.lon| \le 180,\\
o &= \begin{cases} (\bar{s}.lat, \bar{s}.lon), & f \wedge \bar{s} \neq \bot,\\ \bot, & \text{otherwise,} \end{cases}\\
r(t) &= t\pi/180,\\
u(x,y) &= \sin^2 \frac{r(y.lat - x.lat)}{2} + \cos(r(x.lat))\cos(r(y.lat))\sin^2 \frac{r(y.lon - x.lon)}{2},\\
\Delta(x,y) &= 2(6371.0088)\arcsin\sqrt{\min(1, \max(0, u(x,y)))},\\
m_d &= [i \in A : |M(\tau_i, d)| > 0], \qquad \Delta_o(d) = \begin{cases} \Delta(o,d), & o \neq \bot,\\ \bot, & o = \bot. \end{cases}
\end{aligned}
$$

Distances are kilometres; arithmetic is evaluated with the source runtime's floating-point operations. Coordinates represent points, not suburb areas, centre footprints, roads or travel times. Validation enforces numeric finiteness and geographic bounds only; it does not require four decimal places or establish accuracy within 10 metres. Suburb records are approximate centres, and retail records may use store, shopping-centre or suburb points. More decimal places do not prove positional accuracy. Four decimal places correspond to approximately 11.1-metre latitude steps under the model's Earth radius. Accuracy class and GPS accuracy do not expand or contract the radius, and no positional uncertainty interval is computed.

Define the library function

$$
\begin{aligned}
D_o &= [d \in D : \operatorname{valid}(d) \wedge |m_d| > 0 \wedge (o = \bot \vee \Delta(o,d) \le \rho)],\\
\mathcal{N}(D, o, \rho, A) &= \begin{cases} \epsilon, & o \neq \bot \wedge (\neg \operatorname{valid}(o) \vee \neg \operatorname{finite}(\rho) \vee \rho \le 0),\\ \operatorname{sort}_{\prec_s}([(d, \Delta_o(d), m_d) : d \in D_o]), & \text{otherwise.} \end{cases}
\end{aligned}
$$

The screen's stops and reachable item IDs are

$$
Z = \begin{cases} \mathcal{N}(D, o, \rho, A), & \neg f \vee o \neq \bot,\\ \epsilon, & f \wedge o = \bot, \end{cases} \qquad R = \bigcup_{z \in Z} \operatorname{ids}(z.items).
$$

$\prec_s$ compares distance then location ID under $\prec_c$ when either distance is present (absent distance is $+\infty$); when both are absent, it compares location name then ID under $\prec_c$. Remaining ties retain input order. Thus a null library origin bypasses radius validation/filtering, but the screen with filtering enabled and no saved origin has no stops. The radius boundary is inclusive; destination coordinates are always validated. The screen starts with $\rho = 5$ and offers $\{1, 2, 5, 10, 25, 50\}$; it keeps the local radius when switching filter mode and resets it on remount.

For the application's valid selected radii, the task-level nearby predicate is

$$
\begin{aligned}
\operatorname{nearby}(i) \iff i \in A \wedge \big(\omega_i \vee \exists d \in D :\ & \operatorname{valid}(d) \wedge |M(\tau_i, d)| > 0\\
& \wedge (\neg f \vee (o \neq \bot \wedge \Delta(o,d) \le \rho))\big).
\end{aligned}
$$

This is an existential location match followed by a distance test, with an explicit exception for G. No filter removes the origin/distance requirement but retains coordinate validity, recognized-tag eligibility, completion, dismissal and pause rules. `bank` and `errand` tasks remain nearby without any destination or origin, even when filtering is enabled. They are excluded from missing-location and coverage calculations unless an actual matching destination is reachable. Each item is counted once in the reachable ID set even if many locations match it; stop selection is a subsequent operation and cannot remove its nearby membership.

6. Greedy selection, reverse pruning and assignment

Location-coverage errands and unavailable errands are

$$
A_L = [i \in A : \neg\omega_i \vee i.id \in R], \qquad A_\varnothing = [i \in A_L : i.id \notin R].
$$

For each stop, $J(z) = \operatorname{ids}(z.items)$. Starting with $U_0 = R$ and $V_0 = \epsilon$, repeat while $U_k \neq \varnothing$:

$$
\begin{aligned}
c_k(z) &= |J(z) \cap U_k|, \\
z_k &= \operatorname{first}_Z\left(\arg\min_{\prec_s}\{z \in Z : c_k(z) = \max_{w \in Z} c_k(w) > 0\}\right), \\
V_{k+1} &= V_k \parallel [z_k], \qquad U_{k+1} = U_k \setminus J(z_k).
\end{aligned}
$$

Stop if no positive-coverage choice exists. Let the resulting sequence be $V = [v_1, \ldots, v_m]$. Put $V^{(m+1)} = V$; for $j = m, m-1, \ldots, 1$:

$$
V^{(j)} = \begin{cases} \operatorname{delete}_j(V^{(j+1)}), & J(v_j) \subseteq \bigcup_{w \in \operatorname{delete}_j(V^{(j+1)})} J(w), \\ V^{(j+1)}, & \text{otherwise.} \end{cases}
$$

Higher-index deletions leave the next tested position unchanged. Sort the retained stops: $[s_1, \ldots, s_t] = \operatorname{sort}_{\prec_s}(V^{(1)})$. With $B_0 = \varnothing$, assign

$$
\begin{aligned}
e_j &= \operatorname{uniq}_{id}([i \in s_j.items : i.id \notin B_{j-1}]), \qquad B_j = B_{j-1} \cup \operatorname{ids}(e_j), \\
\text{Suggested} &= [(s_j.location, s_j.distance, e_j) : j = 1, \ldots, t], \\
\text{Alternatives} &= \operatorname{sort}_{\prec_s}([z \in Z : z.location.id \notin \{v.location.id : v \in V^{(1)}\}]), \\
\text{Unavailable} &= A_{\varnothing}.
\end{aligned}
$$

Selection counts distinct item IDs, not groups. Pruning uses the current retained stops, in reverse selection order. Assignment uses final display order, keeps source item order within each stop, and assigns each item once. Alternatives retain their full matching item sequences. Input stops are unmodified. This specifies the implemented heuristic, with no global minimum-stop or driving-route objective. The exported planner accepts any supplied errand sequence in place of A_L when computing unavailable items.

7. Groups, pills, previews and counts

For collected errands, group identity is

$$
g(i) = \begin{cases} (\text{list}, \ell.id), & j_i, \\ (\text{item}, i.id), & \neg j_i. \end{cases}
$$

For externally supplied errands without `inheritsListTags`, its fallback is $j_i \iff |H(i.name)| = 0$. Grouping $\Gamma(q)$ gathers rows by g, retaining first group occurrence and row order; distinct list IDs remain distinct even when names agree. Each group has fields `id`, `list`, `rows`, keeping its first row's list record. Group ID pairs encode the source's `list:` and `item:` keys. In group lists, a list group expands and shows a parent-list checkbox iff it has more than one row; otherwise its rows render directly. Inherited pill navigation still opens the list when its group has one row.

Task pills form an insertion-ordered map Π. Scan Z and each stop's item sequence in order: on the first occurrence of an item ID with nonempty $M(\tau_i, d)$, insert $(i, \text{clean}(i.name), \text{head}(M(\tau_i, d)))$. Then scan A: for each optional task insert/overwrite $(i, \text{clean}(i.name), o_i)$, preserving an existing key's position. Thus optional labels overwrite location-derived labels, and unmatched optional tasks append in A order.

$$
\begin{aligned}
N &= \operatorname{ids}(\operatorname{values}(\Pi)) = R \cup \{i.id : i \in A,\ \omega_i\}, \\
T_a &= \operatorname{uniq}([\pi.tag : \pi \in \operatorname{values}(\Pi),\ g(\pi.item) = a]), \\
\text{Pills} &= [(a.id, a.list, a.rows, \operatorname{preview}(a.rows, p), T_{a.id}) : \\
&\qquad a \in \Gamma([i \in A : |T_{g(i)}| > 0])].
\end{aligned}
$$

Here $a.rows$ are all active rows in the retained group, including any sibling that has no individual match. Pill groups follow A order, not stop order; pill tags follow first occurrence in Π.

clean removes every start/whitespace-prefixed `#word` token while preserving the preceding whitespace, collapses whitespace runs to one space, and trims. With $q_{:p}$ its first $\min(p, |q|)$ rows, string concatenation also written $\parallel$,

$$\begin{aligned}
\text{preview}(q,p) &= \text{join}_{\text{\r{u}}}\left([\text{clean}(i.name) : i \in q_{:p},\ \text{clean}(i.name) \neq \text{empty}]\right) \\
&\quad \| \begin{cases} \text{\r{u}} + (|q| - \min(p,|q|))\ \text{more}, & |q| > p, \\ \text{empty}, & |q| \leq p. \end{cases} \\
\text{ChecklistGroups} &= \text{stableSort}_{-\mathbf{1}_{\exists i \in a.rows : i.id \in N}}(\Gamma(E)), \\
\text{PausedGroups} &= \Gamma(P), \qquad \text{LocationGroups} = \Gamma(A_L), \\
\text{CoveredCount} &= |\{a \in \Gamma(A_L) : \forall i \in a.rows,\ i.id \in R\}|.
\end{aligned}$$

The preview consumes p rows before excluding empty cleaned names; the remainder counts rows, not visible names. It never limits matching/planning. Checklist groups with any nearby/optional active task sort first; other group and row order is preserved. Counts for checklist, paused, missing-location lists and coverage are group counts. Stop cards show both $|\Gamma(stop.items)|$ errands and $|stop.items|$ tasks; alternatives count locations. The coverage denominator is $|\Gamma(A_L)|$, and the stop count is |Suggested|.

For an unchecked checklist row, its explanation is selected in this order:

$$\text{reason}(i) = \begin{cases} \text{choose bank/ATM or location yourself}, & \omega_i \wedge i.id \notin R, \\ \text{no match within radius}, & (o \neq \bot \vee \neg f) \wedge i.id \notin N \wedge f \wedge i.id \in W, \\ \text{no matching location in database}, & (o \neq \bot \vee \neg f) \wedge i.id \notin N, \\ \text{empty}, & \text{otherwise}. \end{cases}$$

Completed rows have no such explanation. Missing-location rows use the radius explanation iff $f \wedge i.id \in W$, otherwise the database explanation. Notes show a note mark and no completion checkbox. Only unchecked checklist rows have pause controls; all checklist rows can be dismissed.

8. Checklist and pause transitions

Every durable transition is disabled by the screen when η is true; calling a data mutator directly in history raises the shared read-only error. Read-only local controls remain usable. Unmentioned state components remain unchanged. t denotes the current timestamp.

For a supplied sequence of (id, ϕ) entries, `remember` updates exactly the absent entries or dismissed entries whose saved fingerprint differs:

$$\text{remember}(Q, id, \phi) = \begin{cases} Q[id \mapsto (\phi, \text{false})], & Q[id] = \bot \vee (Q[id].\delta \wedge Q[id].\phi \neq \phi), \\ Q, & \text{otherwise}. \end{cases}$$

After reactive recomputation in a live view, fold `remember` over $[i \in E : \neg d_i]$ using their current fingerprints. Ordinary remembered records are not refreshed on every edit. Automatic membership transactions use `NEARBY_CHECKLIST_MEMBERSHIP_ORIGIN` and create no undo entry. Dismissal sets $Q[i.id] \leftarrow (\phi_i, \text{true})$ for the selected current checklist row. Clear completed applies that assignment to every $i \in E^+$. Neither operation changes original item completion.

`setDone`(id, β) first rejects missing items, notes and headings. If the item's list exists and its original completion is false, remember its current fingerprint before changing the checkbox. Then

$$\begin{cases} i.checks[\text{last}(b(i)).id] \leftarrow \beta, \quad i.updatedAt \leftarrow t, & |b(i)| > 0, \\ i.checked \leftarrow \beta, \quad i.updatedAt \leftarrow t, & |b(i)| = 0 \wedge i.checked \neq \beta, \\ \text{no item change}, & |b(i)| = 0 \wedge i.checked = \beta. \end{cases}$$

Other checkbox values are retained. Setting group completion changes $\ell.done$ and $\ell.updatedAt$ only. Editing records/formatting changes the inputs to ϕ, C, A, E; no explicit undismiss transition is required. A completed dismissed task remains hidden for any fingerprint; an unchecked/note task returns only when the saved and current fingerprints differ and the other eligibility predicates hold.

$$\text{pauseTag}(x,b): \quad P_T \leftarrow \begin{cases} P_T \cup \{n(x)\}, & b, \\ P_T \setminus \{n(x)\}, & \neg b, \end{cases}$$

$$\text{pauseItem}(id,b): \quad P_I \leftarrow \begin{cases} P_I \cup \{id\}, & b \wedge id \text{ exists}, \\ P_I, & b \wedge id \text{ is missing}, \\ P_I \setminus \{id\}, & \neg b. \end{cases}$$

Tag inputs must normalize to nonempty ASCII word strings; item IDs must be nonblank strings. Pauses are separate Yjs map keys with value `true`; reads retain valid true-valued entries and sort keys by $\prec_u$. Resuming an item removes only its individual pause; resuming a tag removes only that tag. Pauses have no expiry and survive edits. No completion/list/original item change is implied.

9. Location, preference and GPS transitions

Starting records require valid numeric coordinates; source in $\{gps, suburb, custom\}$; nonblank label; parseable timestamp string; a present accuracy field that is null (written $\perp$) or finite numeric ≥ 0. Custom sources additionally require a string reference beginning `custom-` with a nonempty suffix. Saving a custom starting point requires the referenced location to exist and be enabled, substitutes its current label/coordinates and sets accuracy to $\perp$. Saving any starting point atomically sets $(s, f) \leftarrow (s_{new}, \text{true})$.

$$\begin{aligned} \text{noFilter}: &(v, \lambda, e, f, \text{showLocations}, \text{showDetails}) \leftarrow (v+1, \text{false}, \text{empty}, \text{false}, \text{false}, \text{false}), \\ \text{suburb}(u): &(v, \lambda, e) \leftarrow (v+1, \text{false}, \text{empty}); \quad \text{saveStart}(u.coords, suburb, u.name, \perp, t), \\ \text{custom}(id): &(v, \lambda, e) \leftarrow (v+1, \text{false}, \text{empty}); \quad \text{saveStart}(X[id].coords, custom, X[id].name, \perp, t, id), \\ &\text{showLocations} \leftarrow \text{false}, \quad \text{showDetails} \leftarrow \text{true}. \end{aligned}$$

Custom selection with a missing/disabled reference instead sets the unavailable-starting-point error and performs no cancellation/save. Clear-start deletes s without changing f. No filter keeps s and ρ. Starting-point tags never filter destinations. The starting-point shortcut sequence is No filter, GPS, h, then enabled custom locations in X order; the editor button follows. Shortcut selection indicators require f and matching source plus suburb label or custom ID. Origin label distinguishes saved GPS, approximate suburb centre and saved custom location; accuracy and update time are display metadata only.

The screen initially has $v = 0$, $\lambda =$ false and empty e. With geolocation available, GPS press increments v, captures $w = v$, sets $(\lambda, e) = (\text{true}, \text{empty})$, and requests one fix with `(enableHighAccuracy=true,timeout=15000 ms,maximumAge=60000 ms)`.

$$\text{gpsSuccess}(w,c,a): \quad w = v \wedge \neg\eta \Longrightarrow \text{saveStart}(c, gps, \text{Last GPS location}, a, t); \ \lambda \leftarrow \text{false},$$

$$\text{gpsError}(w,k): \quad w = v \Longrightarrow \lambda \leftarrow \text{false}; \ e \leftarrow \begin{cases} \text{permission denied}, & k = 1, \\ \text{could not get location}, & k \neq 1. \end{cases}$$

Rejected callbacks do nothing. Unsupported geolocation sets the unsupported-browser error and preserves the saved point. Failures preserve the saved point. Whenever the resolved starting-record/filter pair changes, a reactive effect increments v and clears λ; unmount increments v. There is no background GPS request or continuous position tracking. The saved fix remains usable after movement and reload, with no age-based expiry. The browser may supply a cached fix up to 60 seconds old at request time; the saved timestamp is the callback/save time. Neither that timestamp nor reported GPS accuracy participates in the distance predicate or imposes a 10-metre acceptance threshold. The success guard also checks history; the error callback checks generation only.

Custom locations require unique nonblank IDs/names, valid coordinates, nonempty tag sequences whose strings match optional-`#` ASCII words, optional string address/source/centre, optional Boolean starting-point flag, and optional accuracy class in $\{store, shopping\text{-}centre, suburb\}$. Custom save additionally requires ID prefix `custom-`, trims name and stores normalized first-occurrence-unique tags. The form requires nonblank coordinate fields, uses runtime numeric conversion, splits tags on whitespace/commas, trims address and creates `custom-` plus a UUID for a new location; validation failure leaves stored state unchanged, success clears the form. Copy-coordinates uses o only when present; cancel-edit clears the form.

On saving a selected custom location, atomically update its record and refresh s's label, coordinates and timestamp iff both its old and new starting-point flags are true; otherwise delete s. Deleting a location atomically removes it and deletes s iff s references it; f is unchanged. Reading s also resolves the custom reference, so remote changes

immediately affect origin availability. Enabling a previously disabled location cannot resurrect an old selection. Delete is committed only by the confirm control; cancellation clears the delete target.

Preference reads/defaults and accepted saves are

$$f = (\text{stored filter value} \neq \text{false}), \quad \text{saves require a Boolean},$$

$$p = \begin{cases} \min(p_0, 9), & p_0 \in \mathbb{Z} \cap [1, 50], \\ 3, & \text{otherwise}, \end{cases} \quad \text{saves require } p_0 \in \mathbb{Z} \cap [1, 9],$$

$$h = \begin{cases} h_0, & h_0 \text{ is a unique sequence of known suburb IDs}, \\ [id(\text{Melbourne}), id(\text{Brunswick}), id(\text{Richmond})], & \text{otherwise}. \end{cases}$$

Saving h validates uniqueness and known IDs and leaves s, f unchanged. Legacy imported preview limits $10 \dots 50$ are accepted but read/export as 9.

10. Hashtag browser, suburb editor and navigation

For every $a \in K$, the hashtag count is $c_a = |\{d.id : d \in D, a \in S(d)\}|$; only optional tags may be added with zero count. The complete tag sequence sorts by $\prec_c$. Search reduces the normalized query $n(q)$ to ASCII word characters, then retains tags containing that substring. With nonblank raw query or Show all enabled, display all search matches; otherwise display the first twelve under $(-c_a, a_{\prec_c})$. Selected-tag locations are all matching destinations, sorted by $(name_{\prec_c}, address_{\prec_c})$, absent address treated as empty, with stable remaining ties. Selecting a tag replaces the panel; close clears selection; pause-all toggles only that tag. No origin/radius/coordinate validation is applied to this browser.

The hot-suburb editor takes a local copy h^* of h. Add appends only an absent known ID and clears query; remove filters out the ID; keyboard move swaps with the adjacent entry if in bounds. Drag starts only for the primary pointer's left button, remembers pointer/source/target IDs, accepts only that pointer, and on release moves the source to the target's original index if both exist and differ. Cancel/lost capture clears drag state. Save validates/stores h^*; Cancel/back discards it. The main starting point is unchanged.

Suburb search normalization α applies NFKD, removes combining characters $0300 \dots 036f$, lowercases, expands word-boundary `mt` with optional period and following whitespace to `mount` , replaces runs outside ASCII letters/digits with a space, then trims. Let $q = \alpha(input)$ and $a = \alpha(suburb.name)$. Fuzzy distance is the following substring/transposition recurrence:

$$D_{0,j} = 0, \qquad D_{i,0} = i,$$

$$D_{i,j} = \min \left\{ D_{i,j-1} + 1, D_{i-1,j} + 1, D_{i-1,j-1} + \mathbf{1}_{q_i \neq a_j}, \begin{cases} D_{i-2,j-2} + 1, & i,j > 1 \wedge q_i = a_{j-1} \wedge q_{i-1} = a_j, \\ +\infty, & \text{otherwise} \end{cases} \right\},$$

$$\mathrm{fd}(q, a) = \min_{0 \leq j \leq |a|} D_{|q|,j}, \qquad \theta(q) = \begin{cases} 0, & |q| < 3, \\ \min(3, \max(1, \lfloor |q|/4 \rfloor)), & |q| \geq 3, \end{cases}$$

$$\mathrm{score}(q, a) = \begin{cases} 0, & q = a, \\ 1, & a \text{ starts with } q, \\ 2, & a \text{ contains } q, \\ 3 + \mathrm{fd}(q, a), & \theta(q) > 0 \wedge \mathrm{fd}(q, a) \leq \theta(q), \\ +\infty, & \text{otherwise}. \end{cases}$$

Empty q or nonpositive result limit gives no results; otherwise retain finite scores, sort by $(score, |name|, name_{\prec_c}, id_{\prec_c})$, and take the result limit (default 8). The picker opens and resets active index on focus/input; blur/Escape close it. An arrow opens a closed picker at index 0, otherwise wraps the active index modulo result count. Enter chooses the active result only when the picker is expanded (open with nonblank raw input); a result click also chooses. Choosing sets query to the suburb name, closes, resets index and invokes the consumer; editor add then clears the query. Clear empties query, closes, resets index, invokes its clear callback and focuses input; focus can reopen it but the empty query keeps results unexpanded.

Navigation for a pill with group a sets

$$(\text{showNearby}, \text{openList}, \text{openItem}, \text{returnTarget}) \leftarrow \begin{cases} (\text{false}, \ell.id, \bot, (\ell.id, \bot)), & a \text{ is a list group}, \\ (\text{false}, \ell.id, i.id, (\ell.id, i.id)), & a \text{ is an item group}, \end{cases}$$

returnNearby : $(\text{showNearby}, \text{openList}, \text{openItem}, \text{returnTarget}) \leftarrow (\text{true}, \bot, \bot, \bot)$.

Clicking any individual row opens its list and highlights its item. The return action is supplied only while the open list/item equals the saved target; navigating elsewhere clears that target. The return-link preference defaults true and is stored per user in local storage. If enabled, the return icon is in the list header for a list target and on the highlighted row for an item target. This preference does not sync through the nearby Yjs state.

Directions use $d.address$ iff its accuracy class is `suburb` and its address is nonempty; otherwise use the numeric `latitude,longitude` string. Append its runtime URI-encoded value to `https://www.google.com/maps/dir/?api=1&destination=`. The origin is not included. Distance labels use rounded 1000Δ metres for $\Delta < 1$, otherwise one decimal kilometre. Shopping-centre/suburb accuracy labels affect explanations/directions only. Alternative cards render only while their initially collapsed panel is expanded. Suggestions/alternatives are hidden when $f \wedge o = \bot$ or the location editor is open; the pill strip remains available. With an empty checklist/active sequence, the empty-state message distinguishes remaining paused rows from no eligible tasks.

11. Persistence, history, undo and backups

External sources enter the same mathematical model after conversion to records with stable IDs, names, numeric coordinates and nonempty supported-tag sequences. The source website or dataset does not participate in matching. Bundled catalogues are imported into the frontend at build time and require rebuilding/deploying after changes; custom records are stored in X and can be restored or merged through the backup format. There is no generic in-app importer for arbitrary raw internet datasets. The model covers every valid supplied coordinate worldwide; the bundled retail collection's Melbourne coverage is a property of its input data, not a restriction in the distance algorithm.

$X, Q, P_T, P_I, s, f, p, h$ live in the user's Yjs document: `custom-locations`, `nearby-checklist`, `paused-errand-tags`, `paused-errand-items`, and `nearby-preferences`. Reload/remote updates change the snapshot and recompute the equations. Custom places, pause keys and checklist entries are independent map entries; a starting record is one value so its coordinates/metadata stay together. Concurrent conflict resolution is exactly Yjs's shared map/sequence semantics, not wall-clock timestamp comparison. Undo follows the shared Yjs undo manager; automatic checklist membership is explicitly excluded, while user mutations participate. Historical reads use the selected historical document, and durable mutations are prohibited. The bundled catalogue is shared input data, not copied into user backups.

Nearby backup export is $(X, \bar{s}, f, h, p, Q, P_T, P_I)$ using their read functions. Validate the entire backup before any mutation, including generic records. Nearby validation uses the location/starting/checklist/preview/hot-suburb constraints above; imported paused tags must match optional-`#` ASCII words, paused IDs must be nonblank, filter must be Boolean, checklist keys must be nonempty, and custom IDs must have the required prefix. Invalid import leaves existing state unchanged. A structurally valid custom starting reference need not exist/be enabled at import; subsequent read resolution may yield $\bot$.

For merge/replace mode m, let Y_b denote a supplied backup field. Import custom/checklist maps and pause sets by

$$X' = (\mathbf{1}_{m=merge} X) \oplus X_b, \qquad Q' = (\mathbf{1}_{m=merge} Q) \oplus Q_b,$$
$$P_T' = (\mathbf{1}_{m=merge} P_T) \cup \{n(t) : t \in P_{T,b}\}, \qquad P_I' = (\mathbf{1}_{m=merge} P_I) \cup P_{I,b},$$
$$s' = \begin{cases} s_b, & s_b \text{ is non-null and supplied}, \\ \bot, & m = replace \vee s_b \text{ is explicitly null}, \\ s, & \text{otherwise}, \end{cases}$$
$$y' = \begin{cases} y_b, & y_b \text{ supplied}, \\ \bot, & m = replace, \qquad y \in \{f, h, p\}. \\ y, & \text{otherwise}, \end{cases}$$

$\mathbf{1}_{m=merge} Y$ means keep Y in merge mode, otherwise start empty; $\oplus$ upserts entries with backup values winning. Missing map/set fields contribute empty collections. Imported custom records/tags are stored as supplied; normalization applies when reading/matching or explicitly saving. Missing preferences in replace mode are deleted, so read defaults apply. Importing a starting point does not implicitly enable filtering; s, f are independent import

fields. Merge of a legacy backup preserves absent nearby fields; replace clears them. These mutations share the import transaction and are undoable through the existing application mechanism.

12. Source correspondence

Definitions 2 and 5: `tags.ts`, `retailLocations.ts`, `suburbLocations.ts`, `startingLocation.ts`. Definitions 3 and 6: `hierarchy.ts`, `nearbyErrands.ts`, `data.ts` (`isItemDone`, effective archiving). Definition 4 and transitions 8: `nearbyChecklist.ts`, `data.ts` (fingerprint, membership, dismissal, pauses, completion). Definition 7: `errandGroups.ts`, `nearbyPills.ts`, `NearbyErrandsScreen.svelte`. Transitions 9: `data.ts`, `hotSuburbs.ts`, GPS/selection handlers in the screen. Definition 10: screen, `HotSuburbsScreen.svelte`, `SuburbPicker.svelte`, `suburbSearch.ts`, nearby navigation in `HomeScreen.svelte`/`ListScreen.svelte`, `settings.svelte.ts`. Definition 11: nearby backup handlers in `data.ts` and shared `yjsStore.svelte.ts`.